Technical Design Document

FDE Technical Design Document

Project Name: Support Ticket Router & Assistant

FDE Lead(s): Google Cloud FDE Team (buddy/manager review)
Last Updated: June 13, 2026
Status: Under Review

Executive Summary

Global Retail Hub requires a scalable, highly resilient customer support routing and automated resolution system. Incoming customer messages must be classified by sentiment and intent, routed to the appropriate queues, resolved using grounded FAQ knowledge when possible, and persisted securely in a database.

This Technical Design Document outlines the architecture for the **Support Ticket Router & Assistant**, an agentic system that:

1. **Performs Sentiment-Aware Routing**: Routes tickets to human queues or automated agents based on sentiment and urgency, improving customer experience for frustrated users.
2. **Maintains System Resilience**: Implements a Circuit-Breaker pattern in FastAPI to handle Agent Platform (fka Vertex AI) Agent Search rate limits (HTTP 429) gracefully, preventing system collapse.
3. **Persists Ticket State**: Integrates BigQuery / Cloud SQL to track conversation history and ticket statuses.

Core "North Star" Metrics

- **Routing Accuracy**: >=95% correct classification of "Urgent/Frustrated" tickets vs. standard queries.
- **Circuit-Breaker Sensitivity**: The circuit breaker must open within 3 consecutive HTTP 429 failures and recover within 30 seconds of downstream recovery.
- **Database Write Latency**: p95 database write latency <= 100ms for ticket state updates.
- **FAQ Answer Accuracy**: >=85% Grounding Recall when querying the Twitter support database.

System Architecture

High-Level Diagram

```
None
graph TD
    User([Customer]) -->|1. Submit Message| FE[React Frontend - TypeScript/Vite]
    FE -->|2. Asynchronous API Request| BE[FastAPI Backend - Cloud Run]

    subgraph Backend [FastAPI Backend - Secure Sandbox]
        BE -->|3. Route & Log Request| DB[(BigQuery / Cloud SQL)]
        BE -->|4. Forward to Router| RA[Router Agent - Gemini 2.5 Flash]

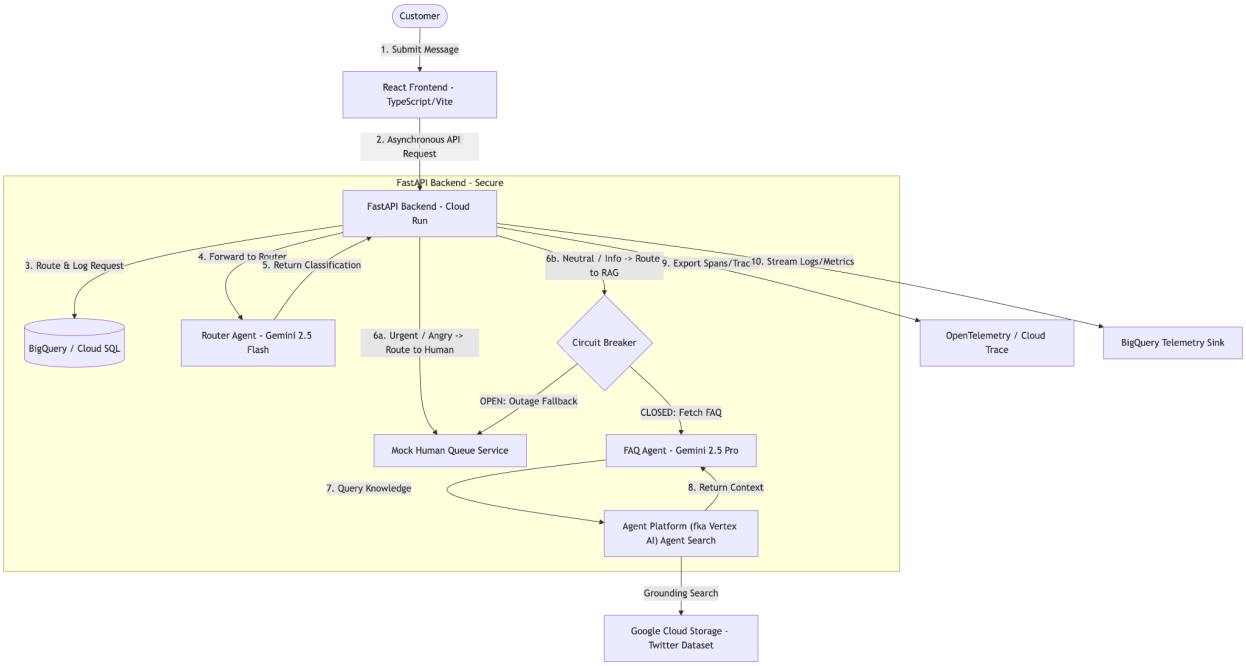
        RA -->|5. Return Classification| BE

        %% Routing decisions
        BE -->|6a. Urgent / Angry -> Route to Human| HQ[Mock Human Queue Service]
        BE -->|6b. Neutral / Info -> Route to RAG| CB{Circuit Breaker}

        CB -->|CLOSED: Fetch FAQ| FA[FAQ Agent - Gemini 2.5 Pro]
        CB -->|OPEN: Outage Fallback| HQ

        FA -->|7. Query Knowledge| RAG[Agent Platform (fka Vertex AI) Agent
Search]
        RAG -->|8. Return Context| FA
    end

    RAG -->|Grounding Search| GCS[Google Cloud Storage - Twitter Dataset]
    BE -->|9. Export Spans/Traces| OTEL[OpenTelemetry / Cloud Trace]
    BE -->|10. Stream Logs/Metrics| BQ[BigQuery Telemetry Sink]
```



Architecture Principles

- **Statefulness**: BigQuery / Cloud SQL serves as the single source of truth for ticket lifecycles (New, In-Progress, Escalated, Resolved).
- **Resilience & Fault Tolerance**: The system operates defensively. GenAI services (which are subject to rate limits and network latency) are wrapped in circuit breakers to ensure the core application remains active.
- **Data Privacy**: Customer PII (emails, names, phone numbers) must be masked in the gateway before reaching the routing agent.

Technical Components & Agent Logic

1. Sentiment Router Agent (ADK)

- **Logic**: Classifies inbound text messages.
- **Sentiment Classification**:
    - **Urgent/Frustrated**: Contains language indicating anger, severe dissatisfaction, or urgency (e.g., "very upset", "late shipping", "charge error").
    - **Neutral/Factual**: Contains standard inquiries (e.g., "What is your return policy?", "Do you ship to Germany?").
- **Prompt**:

```
None
System Instructions:
Analyze the incoming customer support message. Classify its sentiment as either
URGENT_FRUSTRATED or NEUTRAL_FACTUAL.
Extract entities: Order ID, Product Name, Email.
Return output in strict JSON format matching the schema.
```

2. FAQ Agent (ADK)

- **Logic**: Invoked for `NEUTRAL_FACTUAL` queries. Queries the Agent Platform (fka Vertex AI) Agent Search and compiles a grounded answer.
- **Grounding**: Grinds against the Twitter support dataset.

3. Circuit-Breaker Implementation

- **Middleware**: FastAPI middleware monitors the HTTP responses from the `FAQ Agent` and the Agent Platform (fka Vertex AI) Agent Search.
- **State Machine**:
  - **CLOSED**: Normal operations. All FAQ queries are routed to Agent Platform (fka Vertex AI).
  - **OPEN**: Triggered after 3 consecutive failures (HTTP 429 or 503). All FAQ queries bypass Agent Platform (fka Vertex AI) and are routed to the `Mock Human Queue Service` with a predefined auto-reply ("We are experiencing high volume, a human agent is reviewing your request...").
  - **HALF-OPEN**: After a cooldown period (e.g. 30 seconds), the circuit allows a single test query. If successful, it switches back to `CLOSED`. If it fails, it returns to `OPEN`.

4. Database Schema (SQLAlchemy/PostgreSQL)

- **Ticket Table**:

```sql
SQL
CREATE TABLE tickets (
    id UUID PRIMARY KEY,
    customer_id VARCHAR(255) NOT NULL,
    current_status VARCHAR(50) NOT NULL, -- NEW, ESCALATED, RESOLVED
    sentiment VARCHAR(50) NOT NULL,       -- NEUTRAL, ANGRY
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- **Message Table**:

```sql
CREATE TABLE messages (
    id UUID PRIMARY KEY,
    ticket_id UUID REFERENCES tickets(id),
    sender VARCHAR(50) NOT NULL,         -- CUSTOMER, AGENT, SYSTEM
    message_text TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tooling & External Integrations

Database Integration

- **BigQuery / Cloud SQL**: Provisioned within the Sandbox VPC. Connection pooling managed via Python's `SQLAlchemy`.
- **Secret Manager**: Stores the database credentials securely. The application fetches the credentials at startup using IAM-authenticated service accounts.

Agent Platform (fka Vertex AI) Agent Search

- **Setup**: The Twitter customer support CSV dataset is processed, cleaned of empty rows, and indexed into the Agent Search corpus using the standard Agent Platform (fka Vertex AI) embedding model (`text-embedding-004`).

Infrastructure, Security, & IAM

GCP Project Structure

- **Dev Project ID**: <project-name>
- **Region**: us-central1
- **Database Target**: Cloud SQL (PostgreSQL, 1 vCPU, 3.75 GB RAM sandbox instance).

User Authentication (AuthN)

- **Identity Provider**: Google Cloud Identity
- **Access Patterns**: Protect public endpoints using Identity-Aware Proxy (IAP).

Authorization (AuthZ)

- **Service Accounts**: Custom service account (`support-router-sa@<project-name>.iam.gserviceaccount.com`) granted `CloudSQL Client`, `Discovery Engine Viewer`, and `Secret Manager Secret Accessor` roles.

Data Protection & Compliance

- **VPC Service Controls (VPC-SC)**: Restricts database access and model calls within the Sandbox service perimeter.
- **PII Redaction**: Model Armor is configured to redact Credit Card numbers, Social Security Numbers, and email addresses from customer queries before sending to Gemini.

Data Engineering & Intelligence

Data Ingestion

- **Dataset**: Support Customer Support Kaggle Dataset.
- **Ingestion Script**: parses the CSV file, map headers, and uploads chunks to GCS, syncing to the Agent Search.
- **Chunking**: Chunks of 300 tokens, structured by QA pairs (input tweet mapped to response tweet).

Testing & Evaluation Framework

1. Circuit Breaker Simulation Test

- **Test**: `test_circuit_breaker_behavior`
- **Logic**: Mocks the Agent Platform (fka Vertex AI) Agent Search client to throw HTTP 429 on request.
    1. Sends 3 queries.
    2. Asserts that the first 3 return 429 or trigger normal routing.
    3. Sends a 4th query.
    4. Asserts that the 4th query bypasses the RAG call entirely (captured via mocks) and successfully routes to the human queue fallback.

2. Database Integration Test

- **Test**: `test_db_persistence`
- **Logic**: Asserts that tickets are written to the database with the correct sentiment flags, and that transaction rollbacks occur on write failures.

3. Statistical & Heuristic QA Evaluations (CI/CD Gates)

These tests execute on every pull request using a golden dataset of 100 customer support tickets:

- **Router Classification Matrix**: Compares the router's assigned queues against human labels. Computes Precision, Recall, and F1-score for each queue. The CI pipeline fails if **F1-score for 'Urgent/Frustrated' falls below 95%**.
- **BLEU & ROUGE-L Scores**: For automated FAQ replies, calculates ROUGE-L and BLEU scores comparing the generated reply against the historical agent response in the Twitter dataset. Enforces a threshold of **ROUGE-L >= 0.35** and **BLEU >= 0.30**.

4. Semantic QA Evaluation (Nightly/Ad-hoc)

- **Test**: `test_faq_grounding_relevance`
- **Logic**: Nightly runner checks the 100-ticket golden dataset using Gemini 3.5 Flash as an independent judge. Asserts RAG response faithfulness (checking for hallucinations or agent policy violations) is >=90%.

Analytics, Insights & Feedback

User Behavior & Engagement

- **User Actions**: Logs when support agents accept escalated tickets, and SREs monitor circuit-breaker triggers.
- **Session Metrics**: Tracks ticket resolution rate, average escalation rate, and average conversation turn counts.

Operational & Business Intelligence

- **Usage & Cost**: Tracks model execution costs, BigQuery data queries, and Cloud SQL storage metrics.
- **Performance Trends**: Maps response latency, specifically measuring the overhead introduced by database operations and Model Armor filtering.

- **BI Dashboards**: Looker dashboards visualizing Circuit Breaker triggers, average MTTR (Mean Time to Resolve), and the percentage of query deflection.

Observability & Audit

- **Logging**: Structured JSON logging outputted to Google Cloud Logging.
- **Audit Trails**: Security audits log all database access, Secret Manager reads, and manual ticket status changes.